

Lab 3 – Searching & Sorting

Exercise 1 – Merge

2. Worst-case Complexity Analysis

The implemented merge function iterates through two sorted subarrays using two indices. Each element is examined exactly once and appended to the result list.

For an input of size n :

- The array is recursively divided into two halves.
- This division continues until subarrays of size 1 are reached.
- The depth of recursion is $\log_2 n$.

At each recursion level:

- All n elements are processed during merging.
- Each merge step performs at most n comparisons.

Thus, total work:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$T(n) = O(n \log n)$$

This directly reflects the written code because:

- The recursive calls generate $\log n$ levels.
- Each level merges all n elements once.

3. Manual Execution

Input: Original array: [8, 42, 25, 3, 3, 2, 27, 3]

Left Array: [8, 42, 25, 3] - Right Array: [3, 2, 27, 3]

Left Branch :

Left Array: [8, 42] - Right Array: [25, 3]

After merging indices 0-0 and 1-1: [[8, 42], 25, 3, 3, 2, 27, 3]

After merging indices 2-2 and 3-3: [8, 42, [3, 25], 3, 2, 27, 3]

After merging indices 0-1 and 2-3: [[3, 8, 25, 42], 3, 2, 27, 3]

Right Branch :

Left Array: [3, 2] - Right Array: [27, 3]

After merging indices 4-4 and 5-5: [3, 8, 25, 42, [2, 3], 27, 3]

After merging indices 6-6 and 7-7: [3, 8, 25, 42, 2, 3, [3, 27]]

After merging indices 4-5 and 6-7: [3, 8, 25, 42, [2, 3, 3, 27]]

After merging indices 0-3 and 4-7: [2, 3, 3, 3, 8, 25, 27, 42]

Sorted array :

Sorted array: [2, 3, 3, 3, 8, 25, 27, 42]

Splits: 3 - Merges: 7

4. Consistency with Complexity

There are $\log_2 8 = 3$ levels of recursion. And Merge and Sort 8 arrays of size [1] giving use $8 \log_2(8) = 24$

$$8 \times 3 = 24$$

Sorted array: [2, 3, 3, 3, 8, 25, 27, 42]

Splits: 3 - Merges: 7

This matches the $O(n \log n)$ prediction.

Exercise 2 – Performance

1–2. Testing Method

Both bubble sort and quicksort were tested on 20 increasing input sizes.

For each size:

- Best case
- Worst case
- Average case (random input)

Definitions differ:

- Bubble Sort:
 - Best: already sorted
 - Worst: reverse sorted
 - Average: random
- Quicksort (first element pivot):
 - Best: balanced partitions (Even amount of elements in left and right)
 - Worst: already sorted (one partition size 0 and one partition size n)
 - Average: random (partitions between size $[0,n]$ and $[n/2,n/2]$)

3. Performance Discussion

- Bubble sort marginally faster for very small inputs ($n < 20$).
- Quicksort dominates as n increases.
- Worst-case bubble sort grows quadratically.
- Average quicksort follows $n \log n$ trend.

Ascending Order

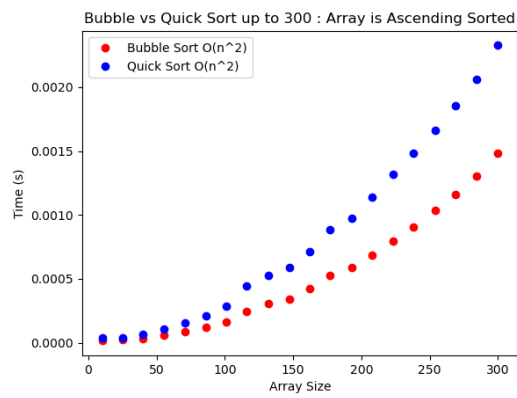


Figure 1: n = 300

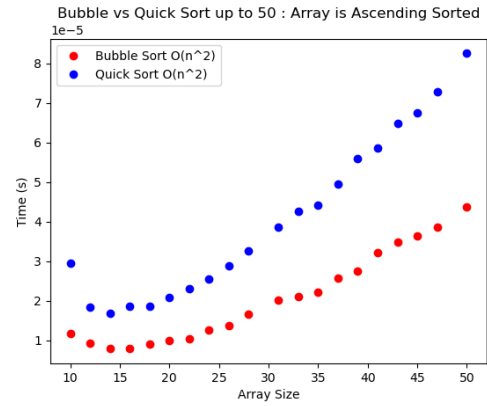


Figure 2: n = 50

Descending Order

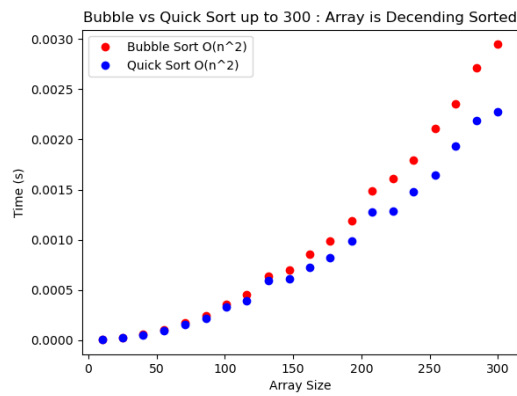


Figure 3: n = 300

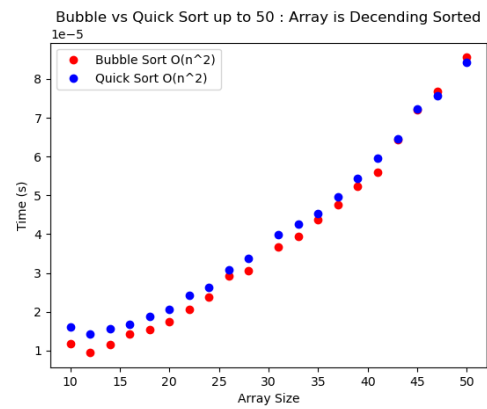


Figure 4: Second image

0.1 Unordered

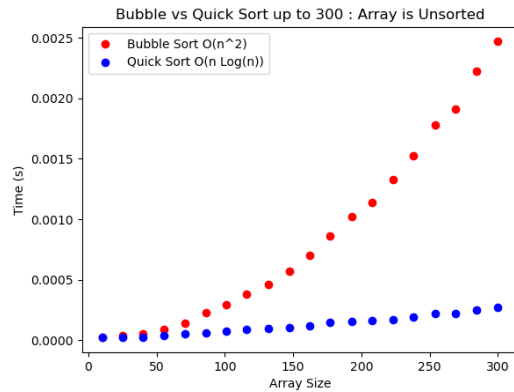


Figure 5: $n = 300$

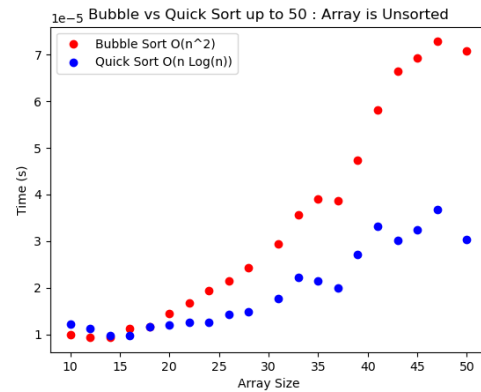


Figure 6: $n = 50$

4. Threshold Choice

Plots show intersection around $n \approx 20$. Based on the plots, a reasonable threshold is:

$$n \leq 20$$

- For $n < 20$, bubble sort is comparable.
- Beyond that, quicksort clearly outperforms.

unless there is a reason not to use Quicksort(eg: large number of elements with a small span of values) it should be used over bubble sort

Exercise 3 – Complexity Analysis

1. Deriving Formulas

Comparisons

Bubble sort uses nested loops:

$$\begin{aligned} & \sum_{i=1}^{n-1} (n-i) \\ &= (n-1) + (n-2) + \cdots + 1 \\ &= \frac{n(n-1)}{2} \end{aligned}$$

Thus:

$$\text{Comparisons} = \frac{n(n-1)}{2} = O(n^2)$$

Average Swaps

In the average case, about half of comparisons result in swaps assuming random data where i is the index such that $[i] \geq [i+1]$ has the same probability as $[i] \leq [i+1]$:

$$\begin{aligned} \text{Average swaps} &\approx \frac{1}{2} \cdot \frac{n(n-1)}{2} \\ &= \frac{n(n-1)}{4} \\ &= O(n^2) \end{aligned}$$

4.Results

Plots show:

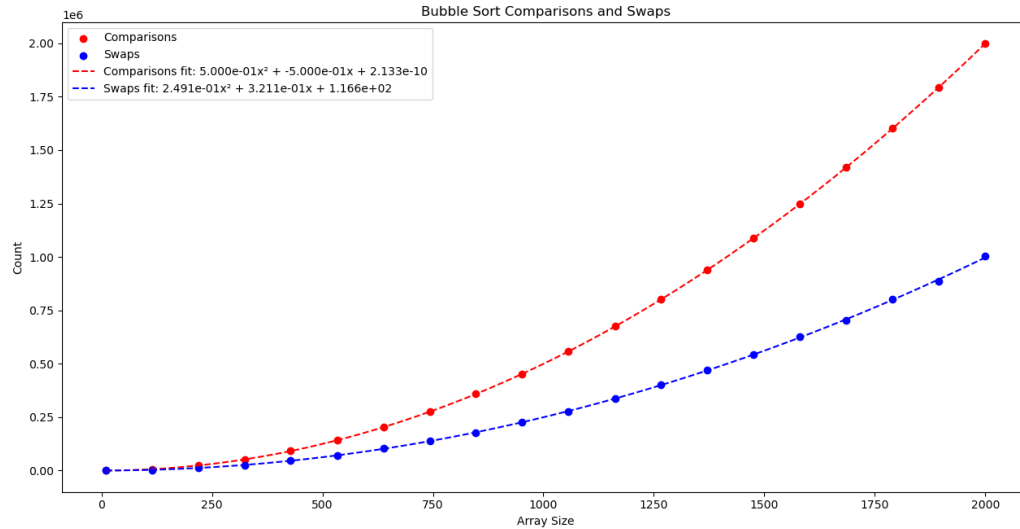


Figure 7: Swaps Vs Comparisons

- Comparisons closely follow $\frac{n(n-1)}{2}$
- Swaps follow approximately $\frac{n(n-1)}{4}$

Conclusion:

Timing data matches theoretical complexity.

Bubble Sort: $O(n^2)$

Exercise 4 - More Complexity Analysis

We consider Quicksort where the pivot is chosen as the **middle index** element of the current subarray.

1. Worst-case complexity derivation

In the worst case, the pivot chosen (middle index element) is already in the correct position and the left and right partitions are already sorted, so the i and j indices meet at the pivot. This partitions the array into two subarrays of size:

$$\left[0 : \frac{n}{2} - 1\right] \quad \text{and} \quad \left[\frac{n}{2} + 1 : n - 1\right]$$

Meaning We will perform $n-1$ splits + $k - n$ comparisons Where $T(n-1)$ is the recursive call on the remaining $n-1$ elements, and the partition step requires $k-n$ comparison operations.

Expanding the recurrence:

$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + (n-1)$$

$$T(n-2) = T(n-3) + (n-2)$$

$$\vdots$$

$$T(1) = 1$$

Summing these terms

$$T(n) = n + (n-1) + (n-2) + \cdots + 1$$

Which simplifies to:

$$T(n) = \frac{n(n-1)}{2}$$

Thus, the worst-case time complexity is:

$$T(n) = O(n^2)$$

2. Example vector of 16 elements

An example of a vector where the pivot is always already in the correct position (causing worst-case partitioning) is:

Given the array:

$$[1, 2, 3, 4, 5, 6, 7, \mathbf{8}, 9, 10, 11, 12, 13, 14, 15, 16]$$

Step 1: Pivot = 8 comparison = 16 After partitioning:

$$[1, 2, 3, 4, 5, 6, 7] \mid \mathbf{8} \mid [9, 10, 11, 12, 13, 14, 15, 16]$$

Step 2 (left subarray): Pivot = 4 comparisons = 7. After partitioning:

$$[1, 2, 3] \mid \mathbf{4} \mid [5, 6, 7]$$

Step 3 (left subarray): Pivot = 2 comparisons = 3. After partitioning:

$$[1] \mid \mathbf{2} \mid [3]$$

Step 4 (right subarray): Pivot = 6 comparisons = 3. After partitioning:

$$[5] \mid \mathbf{6} \mid [7]$$

Step 5 (right subarray): Pivot = 12 comparisons = 7. After partitioning:

$$[9, 10, 11] \mid \mathbf{12} \mid [13, 14, 15, 16]$$

Step 6 (left subarray): Pivot = 10 comparisons = 3. After partitioning:

$$[9] \mid \mathbf{10} \mid [11]$$

Step 7 comparisons = 4: Pivot = 14 (middle index of the right subarray). After partitioning:

$$[13] \mid \mathbf{14} \mid [15, 16]$$

Step 8 (right subarray): Pivot = 15 comparisons = 2. After partitioning:

$$[] \mid \mathbf{15} \mid [16]$$

At this point, every subarray is of size 0 or 1, and the sorting process is complete. The sorted array is:

$$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]$$

Taking a total of 16 split operations 45 comparison operations to sort the sorted array

3. Benchmarking

We run Quicksort on sorted input arrays of increasing size (e.g., $n = 200, 400, 800, 1600, 2000, 2500, 5000$).

each set is run with a set fixed interval of random numbers. (e.g., $[0 : 0], [0 : 1], [0 : 2], [0 : 3], [0 : 4], [0 : 10], [0 : \infty]$)

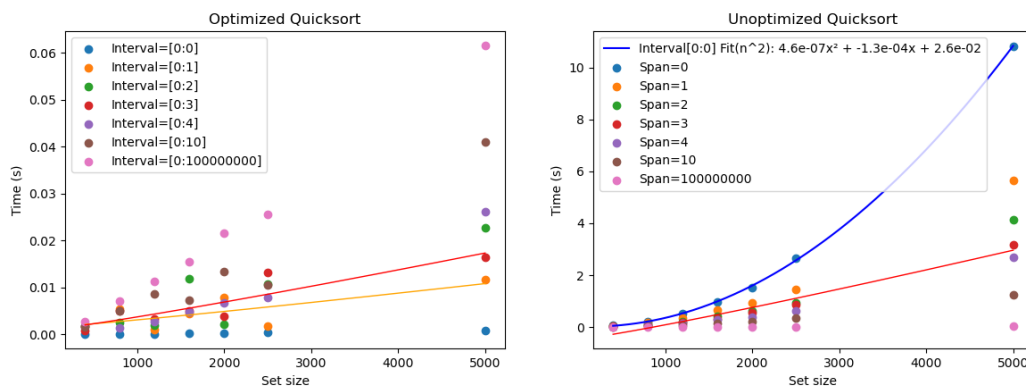
For each input size, we measure the execution time of the algorithm. As the ratio of

$$\frac{vectorsize}{setinterval} \rightarrow 0$$

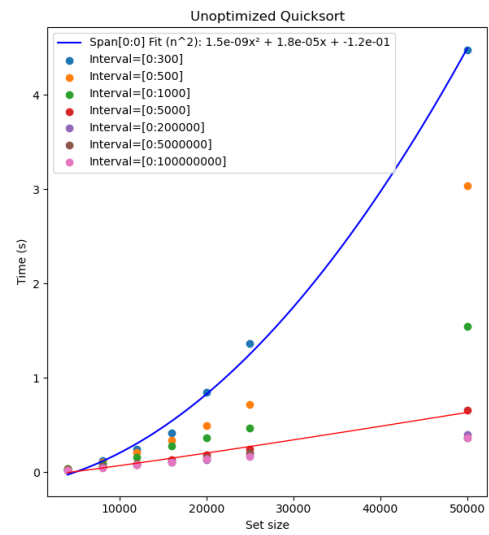
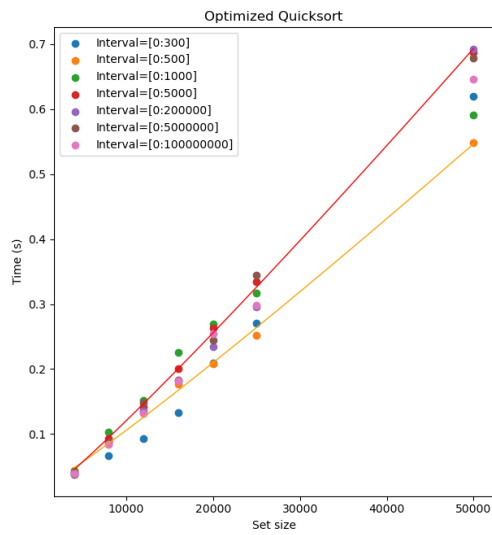
the Time complexity trends from:

$$n^2 \rightarrow n \log n$$

Small Interval, Small N



Large Interval, Large N



4. discussion

We plot execution time versus input size vs interval size and compare it with the function:

$$f(n) = cn \log n \text{ and } c(n^2)$$

for some constant c .

The empirical results follow a trend from n^2 to $n \log n$ confirming the theoretical analysis derived above.